

PROMPTS UTILIZADOS NO NCAS

Núcleo Cognitivo da Colônia Aurora Siger (NCAS)

Documento técnico referente à engenharia de prompts empregada no protótipo desenvolvido em Python, com integração ao modelo Qwen3 por meio do Ollama.

1. Objetivo dos prompts no NCAS

Os prompts utilizados no NCAS têm a função de orientar a simulação de um assistente inteligente responsável por interpretar dados operacionais da colônia. O sistema utiliza informações armazenadas em JSON, aplica previamente uma validação lógica em Python e, quando há risco identificado, envia os dados do módulo ao modelo Qwen3. Dessa forma, a engenharia de prompts estabelece o papel do assistente, o contexto analisado, as restrições de resposta e o formato esperado da saída.

2. Prompt Zero-Shot

Prompt utilizado:

Você é o assistente operacional da colônia Aurora Siger.
Analisar os dados fornecidos e identificar possíveis riscos operacionais.
Apresentar um resumo e uma recomendação de ação.

Finalidade e funcionamento: o prompt solicita diretamente uma análise operacional sem fornecer exemplos de entradas e respostas anteriores. A tarefa é definida pelas instruções: identificar riscos, resumir a situação e recomendar uma ação. Essa abordagem demonstra o conceito de zero-shot prompting, no qual o modelo recebe a tarefa sem exemplos prévios.

Por que foi escolhido: por ser simples e adequado para uma primeira análise dos dados. Também permite demonstrar, no projeto, como instruções explícitas podem orientar um modelo mesmo sem exemplos de referência.

3. Prompt Few-Shot

Prompt utilizado:

Você é o assistente operacional da colônia Aurora Siger.

Exemplo:
Entrada: módulo com integridade de 70%.
Saída: risco identificado devido à baixa integridade.

Agora analise os dados dos módulos da colônia
e identifique situações que exigem atenção.

Exemplo fornecido: uma entrada com integridade de 70% é associada à saída “risco identificado devido à baixa integridade”. O exemplo estabelece o padrão esperado para interpretar uma condição operacional.

Finalidade: orientar o modelo por meio de um caso de referência. Diferentemente do zero-shot, o modelo recebe um exemplo que relaciona uma característica do módulo a uma interpretação de risco. Essa técnica foi escolhida para aumentar a clareza da tarefa e demonstrar o uso de few-shot prompting exigido pela atividade.

4. Prompt com Structured Output

Prompt utilizado:

Analisar os dados operacionais da colônia e responder utilizando a seguinte estrutura:

```
{
  "prioridade": "",
  "risco": "",
  "resumo": "",
  "recomendacao": "",
  "acao_humana": ""
}
```

Estrutura esperada: os campos representam a prioridade do atendimento, a existência ou descrição do risco, um resumo da situação, a recomendação do assistente e a ação que deverá permanecer sob responsabilidade humana.

Vantagem para o sistema: a saída estruturada facilita o processamento pelo Python, pois o resultado pode ser convertido em um dicionário com `json.loads()`. Isso torna a resposta mais previsível e adequada à organização dos dados do NCAS, além de atender diretamente ao requisito de Structured Outputs.

5. Prompt utilizado na integração com Ollama/Qwen3

Estrutura do prompt de integração:

Você é o assistente operacional da colônia Aurora Siger.

Analise os dados do módulo fornecidos pelo sistema NCAS.

IMPORTANTE:

- O sistema Python já identificou se existe risco.
- Não altere o resultado da análise lógica.
- Não invente informações que não estejam nos dados.
- Produza uma resposta objetiva.
- Não apresente seu raciocínio interno.
- Retorne somente um objeto JSON.

Dados do módulo:

Nome, categoria, setor, status, nível de importância, consumo, capacidade, temperatura, pressão, integridade e falha crítica.

Resultado da validação lógica do NCAS:

Risco identificado: [resultado booleano]

Responda utilizando exatamente esta estrutura:

```
{
  "modulo": "",
  "prioridade": "",
  "risco": "",
  "resumo": "",
  "recomendacao": "",
  "acao_humana": ""
}
```

Contexto enviado: o sistema fornece ao modelo os principais atributos do módulo carregado de dados_colonia.json, incluindo nome, categoria, setor, status, nível de importância, consumo, capacidade, temperatura, pressão, integridade e indicação de falha crítica.

Dados do módulo e validação: antes da chamada ao modelo, a função validar_alerta() calcula três condições: falha crítica, status igual a “alerta” e integridade abaixo de 80%. O resultado é combinado pela expressão RISCO = falha_critica OR status_alerta OR integridade_baixa. O valor booleano resultante é enviado no prompt como parte do contexto.

Instruções para a IA: o prompt delimita o papel do modelo, proíbe a alteração do resultado lógico, impede a criação de informações não presentes nos dados, solicita objetividade e determina uma saída somente em JSON. A instrução para não apresentar o raciocínio interno mantém a resposta voltada ao resultado operacional.

Resultado da integração: o código utiliza chat(model='qwen3:4b', ...) para enviar a solicitação ao Ollama. Em seguida, o conteúdo retornado pelo modelo é convertido com json.loads(), permitindo que o NCAS apresente a análise estruturada no terminal e registre a interação no histórico.

6. Estratégia de engenharia de prompts

A estratégia adotada combina diferentes técnicas para demonstrar níveis distintos de orientação do modelo. O zero-shot estabelece a tarefa de forma direta; o few-shot acrescenta um exemplo para indicar o padrão de interpretação; e o structured output define um formato de resposta previsível. Na integração final, essas ideias são complementadas por contexto operacional, instruções explícitas, restrições contra informações inventadas e separação entre a validação lógica realizada pelo Python e a interpretação textual realizada pelo modelo.

Essa separação é importante: a decisão booleana de existência de risco pertence à lógica determinística do sistema, enquanto o modelo é utilizado para organizar, resumir e recomendar ações com base nos dados fornecidos.

7. Relação com os requisitos da atividade

Requisito	Aplicação no NCAS
Zero-shot	Prompt direto para análise de riscos operacionais.
Few-shot	Exemplo de integridade de 70% associado à identificação de risco.
Structured Output	Resposta JSON com campos definidos para análise operacional.
IA generativa	Integração com Qwen3:4b por meio do Ollama.

Requisito	Aplicação no NCAS
JSON	Dados dos módulos são carregados de dados_colonia.json; resposta da IA é processada como JSON.
Regra lógica	Python calcula risco antes da consulta ao modelo.
Simulação/interação	Menu do sistema permite exibir prompts e executar o assistente IA.

A documentação atende especialmente ao requisito 1.5, que solicita a utilização de engenharia de prompts, incluindo exemplos de zero-shot, few-shot e saída estruturada. Também se relaciona aos requisitos do sistema que determinam prompts estruturados e simulação de resposta inteligente.

8. Conclusão

Os prompts desenvolvidos para o NCAS foram projetados para orientar a interação entre os dados operacionais e o assistente inteligente de forma controlada. O uso de zero-shot, few-shot e structured output demonstra diferentes estratégias de Prompt Engineering, enquanto a integração com Ollama/Qwen3 amplia o protótipo para uma interação efetiva com um modelo de linguagem. A combinação entre regras determinísticas em Python e geração textual orientada por prompts contribui para respostas mais consistentes, organizadas e adequadas ao contexto operacional da Aurora Siger.